



Assessment Brief

The assessment is made-up of deliverables as described in the following table:

Assessment Item	Deadline	Extenuating Circumstances/ Student Support Summary Deadline
Group Project (60%): <i>Task 1 (40%)</i> <i>Task 2 (20%)</i>	16 th March	30 th March
Final Exam (40%)	24 th April	11 st May

Group Project (60%)

- ◇ **Group size: 6 students** (one submission per group)

Task 1 (40%): Hex, Memory, and Stack Calculator

Task 1 supports the module's declarative/functional programming goals by encouraging you to structure the solution as a set of small, reusable input-to-output functions (where possible), rather than mixing calculations with input and printing. The menu interface is necessarily imperative, but the core logic should be written in a clear, predictable way with minimal side effects, so that behaviour is easy to reason about and verify. This approach also makes automated testing straightforward, as each function can be checked independently against expected outputs, reinforcing the idea of programs as transformations from inputs to results.

Purpose and overview

This task assesses your understanding through a small, fixed-scope Python programme. You will implement a menu-driven tool that demonstrates:

- Decimal/hex/binary representations (including fixed-width binary)
- Little-endian representation of 16-bit numbers



- Signed vs unsigned 16-bit interpretation (two's complement)
- ASCII storage as bytes and a simple “memory dump”
- Array addressing using offsets (base + index × element size)
- Pointer-style memory access in a toy memory model (write/read bytes at an address)
- A simplified model of stack-frame offsets (bp, bp+2, bp+4)
- A simple register-style view (e.g., AX/BX) to connect values to low-level execution
- Software quality: automated tests + a clear written report with screenshots + a short demo video

This is **not** a freeform project. Every group builds the same programme with the same required options and outputs.

Software and recommended libraries

- **Python version:** Python 3.11+
- **External packages:** Not required. You may complete Task 1 using the **Python Standard Library only**.
- **Testing framework (choose one):**
 - **Option A: unittest** (built into Python; no installation needed)
 - **Option B: pytest** (optional; if you choose it, you install it yourself)

Reference links:

- unittest: <https://docs.python.org/3/library/unittest.html>
- Python Standard Library: <https://docs.python.org/3/library/index.html>
- pytest: <https://docs.pytest.org/>

What you must build

You must create:

- `programA.py`: the actual programme
- `testA.py`: unit tests (either `unittest` or `pytest`)
- `Task1_Report.pdf`: a written report that includes screenshots and a clear, comprehensive explanation of your programme (what you implemented, how each menu option works, and how you verified correctness through testing).
- `README.md`: short instructions for running the programme and tests
- `Task1_DemoVideo.mp4` (or a share link): a short screen-recording demonstrating (1) running the programme and using the menu options, and (2) running the test suite (either `unittest` or `pytest`) and showing the tests passing.



BIRMINGHAM CITY
University

CMP5361 – Computer Mathematics and Declarative Programming
Academic Year 2025-26

Programme menu (must match this numbering)

When you run `programA.py`, it must display:

- 1) Convert (decimal → hex and 16-bit binary)
- 2) Little-endian pack/unpack (16-bit)
- 3) ASCII memory dump
- 4) Array addressing
- 5) Stack frame (bp offsets)
- 0) Exit

The user enters an option (0–5). The programme performs that option, then returns to the menu until the user chooses **0**.

Programme specifications

Option 1: Convert (decimal → hex and binary)

Input: integer n (assume $0 \leq n \leq 65535$)

Output must include these two lines:

- `HEX = <hex value>`
- `BIN(16) = <16-bit binary string>`
- `SIGNED16 = <signed interpretation>`

Rules:

- `BIN(16)` must always be **16 characters** long (pad with leading zeros).
- `SIGNED16` interprets the same 16-bit pattern as **two's complement**:
 - if $n < 32768 \rightarrow \text{SIGNED16} = n$
 - if $n \geq 32768 \rightarrow \text{SIGNED16} = n - 65536$
- Example: $n = 65535 \rightarrow \text{BIN}(16) = 1111111111111111$ and `SIGNED16` = -1

Option 2: Little-endian pack/unpack (16-bit) + memory write/read

Input: integer n ($0 \leq n \leq 65535$) **and** a memory address `addr` (accept decimal or hex like `0x2000`)



BIRMINGHAM CITY
University

CMP5361 – Computer Mathematics and Declarative Programming
Academic Year 2025-26

Output must include:

- LOW BYTE = <0..255>
- HIGH BYTE = <0..255>
- UNPACKED = <number>
- Memory write/read evidence (see below)

Rules:

- LOW BYTE is the **least significant 8 bits** of n .
- HIGH BYTE is the next 8 bits.
- UNPACKED must equal the original input n .
- Write LOW BYTE to memory at **addr**
- Write HIGH BYTE to memory at **addr+1**
- Read both bytes back and print them

Include lines in this style (values filled in):

- `MEM[0x2000] = 0xE8`
- `MEM[0x2001] = 0x03`
- `READ MEM[0x2000] = 0xE8`
- `READ MEM[0x2001] = 0x03`

Option 3: ASCII memory dump + null terminator + length scan

Input: string s (maximum 10 characters)

Output format: one line per stored byte in this form:

`0x1000 : 0xHH`

Rules:

- Base address is **0x1000**
- Each character is stored at the next address (+1)
- You must also store a **null terminator 0x00** after the last character (like a C-style string) and include it in the dump
- After the dump, print: `LENGTH (until 0x00) = <number>`

Option 4: Array addressing + dereference (read/write one element)

Input:



- base (address)
- index (i)
- size (element size in bytes: 1 or 2)
- mode (read or write)
- value (only if mode is write)

Output must include this line:

ADDRESS = base + index*size = <computed address>

Rules:

- If mode = write:
 - store the value into memory at ADDRESS
 - if size=1, store 1 byte
 - if size=2, store 2 bytes in **little-endian**
 - print confirmation of the write
- If mode = read:
 - read 1 byte (size=1) or 2 bytes (size=2) from memory
 - print the value read

Include a clear message in this style:

- WRITE size=2 value=1000 to ADDRESS 0x3004
- READ size=2 from ADDRESS 0x3004 = 1000

Option 5: Stack frame (simplified bp offsets) + register-style view

This is a **toy model** (simplified) to show where parameters would appear relative to a base pointer.

Input: two integers a and b

Assume:

- return address is at bp
- first parameter is at bp+2
- second parameter is at bp+4

Output must show a small table like this style:

```
bp      : RETURN
bp+2    : a = <value>
```



```
bp+4 : b = <value>
```

After the stack table, print a simple register-style view that simulates a CPU using registers:

- AX = <a>
- BX =
- AX (AX+BX) = <a+b>

Notes: you do not need to simulate real memory addresses, just display the offsets and values.

Unit testing¹ requirements

Minimum requirements

Your `testA.py` must contain **at least 10 meaningful tests** and must cover:

- Pack/unpack boundary tests for: **0, 1, 255, 256, 65535**
- At least one ASCII dump test for a known short string (e.g., "A" or "HELLO")
- At least one array addressing test (example: base=1000, index=3, size=2 → 1006)
- At least one stack frame test (checks offsets and printed/returned values)
- At least one conversion test ensuring `BIN(16)` is always 16 bits
- At least one test for `SIGNED16` (e.g., 65535 → -1, 32768 → -32768)
- At least one test that writes to toy memory then reads back (Option 2 or Option 4)

How to structure your code so testing is easy

Menu-based programs are hard to test if everything is inside `input()` and `print()`. To make testing straightforward:

- Put the main calculations into **small functions**
- Keep the menu only for input/output and call your functions
- In `testA.py`, import your functions from `programA.py` and test them

¹ A unit test is code that automatically checks your programme's logic. It feeds a known input to a function and checks the output matches what is expected.



Recommended helper functions (example names):

- `convert(n) -> (hex_str, bin16_str)`
- `pack_u16_le(n) -> (low, high)`
- `unpack_u16_le(low, high) -> n`
- `ascii_dump_lines(s, base=0x1000) -> list[str]`
- `element_address(base, index, size) -> address`
- `stack_frame_lines(a, b) -> list[str]`
- `memory_write(addr, byte) / memory_read(addr) (or equivalent)`

Important: If you use `pytest`, your `README.md` must include how to install and run it.

Written report requirements

This report measures your ability to explain your work clearly and provide evidence of correct behaviour.

Your report must include the following:

Overview (short)

- Describing what your programme does and what each menu option demonstrates.

Evidence via screenshots

Include **at least 10 screenshots** (you may include more). Your screenshots must show:

1. The **main menu**
2. The **output for the menu options** (Options 1–5)
3. The **test output** (running `unittest` or `pytest`, showing tests passing)
4. At least one example of “**memory write + memory read-back**” (Option 2 or Option 4)

Explanation of each option

- What input does the programme ask for?
- What output does it print?
- What computing idea does it demonstrate?

Testing evidence



BIRMINGHAM CITY
University

CMP5361 – Computer Mathematics and Declarative Programming
Academic Year 2025-26

- The exact command you used to run tests (`unittest` or `pytest`)
- Evidence that tests pass (screenshot of output)

Deliverables and submission format

Submit **one zip file per group** containing:

- `programA.py`
- `testA.py`
- `Task1_Report.pdf`
- `Task1_DemoVideo.mp4` **OR** a text file `Task1_DemoVideo_Link.txt` containing a shareable link (if the video file is too large)
- `README.md`

README.md must include:

- Python version used
- How to run the programme: `python programA.py`
- How to run tests (commands for your chosen framework):
 - `unittest: python -m unittest (or python -m unittest -v)`
 - `pytest: python -m pip install pytest then pytest (or pytest -q)`
- If you submit a link instead of a video file: where the link is provided and confirmation that staff can access it.

Marking criteria (40 marks total → 40% of module)

A) Programme correctness: 25 marks

- Menu works correctly (loops, valid options, exits)
- Option 1 Convert correct format + 16-bit binary + SIGNED16
- Option 2 Pack/unpack correct + memory write/read + unpack matches input
- Option 3 ASCII dump correct address + null terminator + LENGTH line
- Option 4 Correct address calculation + read/write behavior based on mode and size
- Option 5 Correct stack-frame offsets/values + register-style view

B) Unit tests: 5 marks

- At least 10 meaningful tests included
- Required boundary/edge cases included
- Includes SIGNED16 test and memory write/read-back test

- Tests run and pass (unittest or pytest)
- Code is structured so logic is testable (functions separated where reasonable)

C) Written report: 10 marks

- Clear overview and correct explanation of concepts
- Required screenshots provided and readable
- Explanations for Options 1–5 are correct and specific
- Testing evidence included and clear



Task 2 (20%): Choose one of the following two options for this task

Option 1: Imperative vs Declarative/Functional Programming in One Language

You will choose one programming language of your choice and explain how it supports imperative and declarative/functional styles. You will also demonstrate how behaviour can be described clearly using Gherkin scenarios, and how the same behaviour can be expressed using simple preconditions and postconditions (at an introductory level; no formal proofs required).

Deliverable

- Task2_LanguageReport.pdf (8-20 pages, one combined submission per group)

Required report structure (6 sections = 6 students)

Your report must contain **six sections**, one per student, clearly labelled with the student's name and role. Each section should be about **1-3 pages** (approx.), keeping the whole report within **8 - 20 pages**.

Section 1: Language overview

Include:

- What the language is commonly used for
- What paradigms it supports (imperative / functional / declarative / logic)
- One short example (5–10 lines) that shows the “style” of the language.

Section 2: Imperative style explanation + example

Explain imperative style in the chosen language (step-by-step instructions, variables, loops, mutation).

Include:

- A short description of what “imperative” means in practice
- One small example (10–20 lines maximum)



Choose **one simple task** (the whole group must use the same task for Sections 2 & 3):

- Sum a list of numbers
- Count vowels in a string
- Find the maximum in a list
- Convert a number to binary (fixed-width)

Section 3: Declarative/functional style explanation + example

Explain how to express the **same task** from Section 2 in a more declarative/functional way.

Include:

- A short description of what “declarative/functional style” means (focus on expressions, functions, minimal mutation)
- One small example (10–20 lines maximum) for the **same task** as Section 2, using one of:
 - recursion, OR
 - map/filter/reduce, OR
 - comprehensions (if appropriate)

Section 4: Types as sets (simple, with examples)

Explain the idea that “a type is a set of possible values” using three examples:

1. **Basic type** example (e.g., boolean has two values)
2. **Subset / constrained type** example (e.g., Percentage is integers 0..100)
3. **Union and product types** (one of each):
 - Union: a value is one of several alternatives (e.g., Shape types)
 - Product: a value is a pair/tuple of components (e.g., Card = Rank × Suit)

You may use clear math notation and/or short pseudocode.

Section 5: Behaviour description using Gherkin

Write a **Gherkin specification** for the chosen task (the same one used in Sections 2 & 3).

Your section must include:

- **One Feature** title and narrative:
 - *As a [user role], I want [feature], so that [benefit]*



- **Two scenarios:**
 1. a normal case
 2. an edge case (e.g., empty list, invalid input, boundary value)
- **One Scenario Outline** with an **Examples** table containing **at least 4 rows**

Keep it simple and testable: each “Then” should be a clear, checkable outcome.

Section 6: Preconditions/postconditions (Hoare Logic)

This is a lightweight correctness view: **write preconditions and postconditions** for the same task (no advanced proof rules required).

Include:

- **Two Hoare-style triples** written as:
 - {Precondition} action {Postcondition}
- A short table comparing imperative vs declarative/functional style (5–8 points)

Important: This is not a formal proof task. Preconditions/postconditions can be written in simple logic-like statements (e.g., “input is a list of integers”, “result equals the sum of all elements”).

Conclusion section: A brief conclusion (half-one page): what the group learned about the language, style, and specifying behaviour.

Expectations

- Keep code snippets short (10–20 lines each)
- Use the same task for Sections 2, 3, 5, and 6
- Make scenarios and pre/post conditions consistent with the examples

Marking criteria (20 marks total → 20% of module)

- **Task 2 weighting:** 20% of the whole module
- **Group size:** 6 students (one submission per group)
- **Structure:** 6 sections in the report (Section 1 - Section 6)
- **Responsibility:** each student is responsible for **one** section
- **Marking approach:** marks are awarded **individually**, based primarily on the quality of the student’s own section



- **Important:** Task 2 remains **20% total** (it does **not** become $20\% \times 6$), in which:
 - Each section (individual) (15%)
 - Conclusion (group) (5%)
- **Overall consistency:** a small adjustment may be applied for coherence/consistency of the combined final report.

Option 2: Algorithm Spotlight (Declarative/Functional Perspective)

Purpose

In this task, your group will choose **one algorithm** from the list below and produce a short report explaining:

- What the algorithm does and why it is useful
- How to represent its data clearly (linking to types/data modelling ideas)
- How the algorithm works step-by-step
- A worked example (input $\rightarrow$ output)
- How the algorithm can be explained in a **declarative/functional style** (as transformations and rules)
- Simple behaviour checks (test ideas)

This is a **structured** task: you must follow the required report structure and division of work.

Algorithm choices (choose ONE)

Pick **one** algorithm for the whole group:

1. **Conway's Game of Life (Cellular Automaton)**
Focus: rule-based transformation "current grid $\rightarrow$ next grid".
2. **Unbeatable Tic-Tac-Toe using Minimax** (optional: alpha-beta pruning, not required)
Focus: game state evaluation and choosing the best move.
3. **Map-Filter pipeline**
Focus: transforming and filtering a list/sequence using rules.
4. **Fold/Reduce (accumulation pattern)**
Focus: combining a list into one result (sum/product/max) using an accumulator.
5. **Predicate-based validation (constraints as boolean functions)**
Focus: validating values such as Percentage 0..100, Rating 0..5, etc.



6. Recursion pattern (factorial OR list processing)

Focus: base case + recursive step; show how recursion expresses the solution.

Deliverable

Submit **one PDF per group**: Task2_AlgorithmSpotlight.pdf (**10 - 20 pages**)

No code submission is required. You may include short pseudocode/code snippets inside the PDF if helpful.

Required report structure (6 sections = 6 students)

Your report must contain **six sections**, labelled **Section 1** to **Section 6**, with the **student's name** responsible for each section.

Section 1: Problem statement

- What problem does the algorithm solve?
- What are the inputs and outputs?
- Give one short real-world example of when it is useful.

Section 2: Data model / representation

- How is the data represented?
- Why is that representation appropriate?
- Provide a tiny example of the data (e.g., a small list, a small grid, a simple game board).

Examples:

- list of numbers, list of records
- set of live cells (Game of Life)
- 3×3 board representation (Tic-Tac-Toe)

Section 3: Algorithm rules / steps (the core)

- Explain the algorithm clearly in steps or rules.
- Keep it understandable: bullet points or numbered steps are fine.
- Mention any important edge cases.



Section 4: Worked example (input → output)

- Show a small example input and the resulting output after the algorithm runs.
- For Game of Life: show the grid at time t and time $t+1$ (or two steps).
- For Minimax: show one board position and explain the chosen best move.

Section 5: Complexity (simple explanation)

- Describe time cost and space cost in plain language.
- You may include Big-O, but it is not required. Clarity matters more.

Section 6: Declarative/functional framing + behaviour checks

Explain how the algorithm can be described as:

- a set of rules / transformations from input to output (declarative thinking)
- using small reusable functions (functional thinking)

Also provide **3 simple behaviour checks** (test ideas), each written as:

- Input:
- Expected output:
- Why this test matters (normal case or edge case)

Marking criteria (20 marks total → 20% of module)

- **Task 2 weighting:** 20% of the whole module
- **Group size:** 6 students (one submission per group)
- **Structure:** 6 sections in the report (Section 1 - Section 6)
- **Responsibility:** each student is responsible for **one** section
- **Marking approach:** marks are awarded **individually**, based primarily on the quality of the student's own section
- **Important:** Task 2 remains **20% total** (it does **not** become $20\% \times 6$)
- **Overall consistency:** a small adjustment may be applied for coherence/consistency of the combined final report

Expectations

- Keep diagrams simple (small grids/boards/examples).



BIRMINGHAM CITY
University

CMP5361 – Computer Mathematics and Declarative Programming
Academic Year 2025-26

- Keep examples small enough to fit on a page.
- Use consistent terminology and formatting across the group.
- You are not assessed on implementing a full program for this task (option 2); this is a structured explanation task.



BIRMINGHAM CITY
University

CMP5361 – Computer Mathematics and Declarative Programming
Academic Year 2025-26

Group Contribution and Individual Roles (mandatory)

Your report must include a section titled: “**Group Contribution and Individual Roles**” which includes:

- A table listing each group member, their responsibilities, and what they delivered (e.g., implementation of Option 2 (Task 1) & Section 3 (Task 2), implementation of Option 3, tests).
- A short paragraph describing how the group collaborated (e.g., meetings, division of tasks, integration process).

Engagement: Any student identified as not engaging or not collaborating (based on this section and/or staff follow-up) will receive **0** for this assessment.



CMP5361 – Computer Mathematics and Declarative Programming
Academic Year 2025-26

Final Exam (40%)...